Model Compression and Efficiency Optimisation for Edge Deployment

Diploma Seminar 2 — Presentation 1

Research Objective, Methodology, and Validation Plan

Hubert Jaczynski

Warsaw, 2026

Warsaw University of Technology

Supervisor: PhD. Maria Ganzha

Agenda

1. Research problem and objective
2. Why the problem matters in edge deployment
3. Methodology and research design
4. Planned models and optimisation methods
5. Planned solution architecture
6. Implementation plan
7. Validation and evaluation strategy
8. Current status and discussion points

Research problem

- Modern object detection models are accurate, but often too heavy for **resource-constrained edge devices**.
- In practice, deployment is limited by:
  - inference latency / FPS,
  - memory footprint and model size,
  - hardware- and runtime-dependent efficiency.
- The key challenge is to improve efficiency **without making detection quality unusable**.

Why edge deployment is difficult

- Edge devices do not only have lower compute than servers. They also have:
  - tighter RAM limits,
  - stricter real-time constraints,
  - device-specific runtime limitations,
  - in some cases power or battery constraints.
- Therefore, a model with good benchmark accuracy may still fail in practice.
- The deployment target is a **full system problem**: model architecture, precision, runtime, and hardware all matter together.

Edge Devices

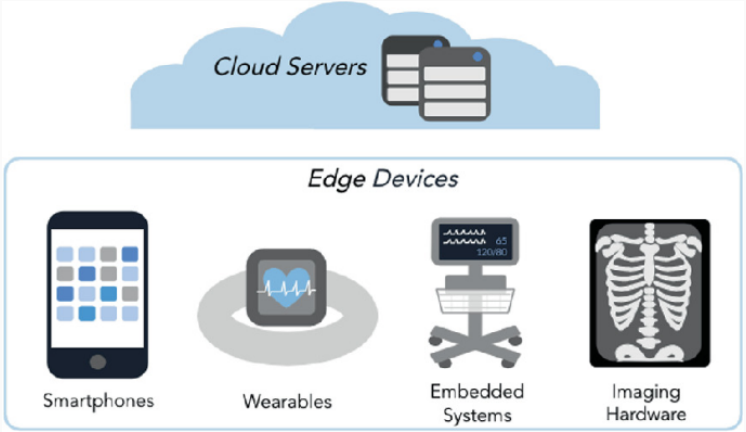

Figure: Examples of edge-device variants [1].

Precise research objective

Main objective
Identify which model compression and efficiency optimisation techniques, or combinations of techniques, provide the best **accuracy–latency–size trade-off** for real-time object detection on constrained hardware.

- **What I am solving:** detectors are often not directly deployable on edge hardware.
- **What I want to achieve:** a reproducible comparison of selected optimisation strategies under realistic deployment conditions.
- **What success means:**
  - a clear evaluation protocol,
  - a compressed or efficient detector that meets a practical latency budget,
  - limited accuracy degradation relative to the baseline,
  - conclusions about which methods work best for the chosen hardware/runtime.

What success means in this thesis

- Success is not only obtaining a smaller model.
- The thesis should produce:
  - a well-justified baseline model,
  - a reproducible optimisation and benchmarking procedure,
  - at least one optimisation path that improves deployment usefulness,
  - conclusions about the trade-offs between compression methods.
- In practical terms, a successful result should show that:
  - latency or FPS improves meaningfully,
  - model size decreases,
  - accuracy degradation remains acceptable,
  - conclusions are based on measurements from the target runtime.

Chosen methodology

- The thesis uses an **experimental comparative methodology**.
- I will compare a baseline detector with multiple optimised variants under **controlled conditions**.
- The methodology is appropriate because the research question is not only theoretical: it asks which method works **best in practice** for a specific deployment setting.
- The study combines:
  - **literature-driven method selection**,
  - **implementation and benchmarking**,
  - **quantitative evaluation** using consistent metrics.

Why this methodology fits the objective

- The objective asks **which methods work best in practice**, so it requires measurable comparison.
- A theoretical analysis based only on parameter count or FLOPs would not be enough.
- The chosen methodology allows me to:
  - test multiple optimisation techniques under the same conditions,
  - observe real deployment behaviour,
  - compare isolated methods and combined pipelines,
  - connect implementation choices directly to thesis conclusions.
- This is especially important because latency gains are often hardware- and runtime-dependent [2].

Planned research design

Candidate models

- one efficient baseline detector,
- one stronger teacher model,
- optimised student variants.

Key parameters

- input resolution,
- quantisation mode: FP16 / INT8 / PTQ / QAT,
- pruning type and pruning ratio,
- distillation temperature and loss weights,
- target device and runtime backend.

Research design: model selection process

- **Step 1:** benchmark unoptimised candidate detectors.
- **Step 2:** choose the main baseline using a Pareto trade-off between accuracy, latency, and model size.
- **Step 3:** if distillation is used, select a stronger model as the teacher.
- **Step 4:** keep the final experiment matrix small enough to remain reproducible and feasible within the thesis schedule.

Why COCO is a useful benchmark dataset

- **MS COCO** is one of the most widely used benchmarks for object detection [3].
- It is useful for this thesis because it provides:
  - a large number of labelled images,
  - many object categories,
  - objects at different scales and in complex scenes,
  - a standard evaluation protocol used in the literature.
- COCO-style evaluation also aligns well with the planned metrics, especially **mAP@[.5:.95]**.
- Even if the final thesis uses a reduced subset or an additional dataset, COCO remains a strong reference point for comparing results with prior work.

Planned baseline and candidate model roles

- The planned workflow distinguishes between **three roles**:
  - a **baseline edge-friendly detector** [4, 5, 6, 7],
  - a **teacher model** with stronger accuracy,
  - one or more **compressed student variants** improved, if needed, by distillation [8].
- The baseline should already be realistic for deployment, because compressing an unsuitable architecture may not produce a useful final system.
- The teacher model does not need to be deployable on the edge device; its purpose is to improve training through distillation [8].
- The final comparison will therefore include both:
  - **architecture-level choices**,
  - **post-selection optimisation choices**.

Candidate architectures for the thesis

- **YOLO-family detectors**
  - strong real-time baseline,
  - widely used for edge-oriented deployment,
  - good candidate for pruning and quantisation experiments.
- **EfficientDet**
  - designed around efficiency/accuracy scaling,
  - useful as an example of an efficient-by-design detector.
- **MobileNet-based SSD-style models**
  - lightweight and simple deployment path,
  - useful as a compact baseline for comparison.
- **DETR-style / RF-DETR models**
  - interesting for comparing modern transformer-based detection with CNN-based real-time detectors,

Representative references: YOLO [4]; EfficientDet [5]; MobileNets [6]; SSD [7]; DETR [9].

Planned optimisation methods

- **Quantisation** [10]
  - FP16 and INT8 as the main practical precision targets,
  - PTQ for fast conversion,
  - QAT when PTQ gives too much accuracy loss.
- **Pruning** [11, 12]
  - mainly structured pruning, because it is more likely to improve real latency on standard hardware.
- **Knowledge distillation** [8]
  - used as a recovery mechanism after compression or as guidance for a compact student model.
- **Efficient-by-design model selection** [13, 5]
  - using published efficient architectures instead of performing a full NAS search.

Brief explanation of methods

- **Post-training quantisation (PTQ):** convert a trained FP32 model to lower precision after training, usually using a calibration subset [10].
- **Quantisation-aware training (QAT):** simulate low-precision effects during training so the model adapts to rounding and clipping [10].
- **Structured pruning:** remove complete channels, filters, or blocks to obtain a smaller dense model [11, 12].
- **Knowledge distillation:** train a smaller student to mimic a larger teacher, often improving compact-model accuracy [8].

Solution architecture

Dataset / annotations

↓

Baseline detector selection

↓

Optimisation modules

quantisation pruning distillation

↓

Export / deployment configuration

runtime, precision, device-specific setup

↓

Benchmarking and evaluation

accuracy, latency, memory, size, optional energy

↓

Comparison and thesis conclusions

Existing components vs thesis contributions

Existing or already started

- literature review,
- thesis problem definition,
- prototype repository structure,
- initial evaluation notebook,
- initial checkpoint assets.

Main thesis contribution

- final detector selection,
- integrated optimisation workflow,
- controlled benchmark experiments,
- deployment-oriented comparison,
- interpretation of trade-offs and recommendations.

Implementation plan

- **Methods and techniques:**
  - structured pruning as the main pruning strategy,
  - PTQ as a fast baseline and QAT as the stronger quantised approach,
  - teacher–student distillation for accuracy recovery,
  - architecture comparison rather than a full NAS search, based on published efficient detector families [4, 5, 6, 7, 9].
- **Computing environment:** Python-based workflow, GPU acceleration when available, edge-like benchmarking on the target runtime.
- **Planned tools:** PyTorch for model development, TorchMetrics.
- **Optional deployment tools:** ONNX Runtime, TensorRT, TFLite, or OpenVINO, depending on which export path is stable enough for the selected model and hardware target.

Planned experiment sequence

1. Select and benchmark candidate baseline models.
2. Fix dataset split, resolution, and evaluation procedure.
3. Establish the reference baseline on the chosen runtime.
4. Apply single optimisations separately:
   - quantisation,
   - pruning,
   - distillation-based compact training if applicable.
5. Test combined optimisation pipelines.
6. Compare results and analyse trade-offs.

Validation and evaluation strategy

- **Primary metrics:**
  - detection accuracy: mAP@[.5:.95], optionally mAP@0.5, following standard COCO-style evaluation [3],
  - latency: model-only and end-to-end time, plus FPS,
  - model size and peak memory usage.
- **Optional metric:** energy per inference, if measurement is feasible on the target platform.
- **Experiments:**
  - baseline vs each single optimisation method,
  - baseline vs combined optimisation pipeline,
  - comparison across deployment backends or devices if possible.
- **Validation rule:** a method is useful only if improvements are demonstrated on the intended runtime, not only by FLOPs or parameter count.

Evaluation criteria and decision logic

- A candidate method will be judged by the balance between:
  - accuracy retention,
  - latency improvement,
  - memory / size reduction,
  - implementation complexity.
- Some methods may improve model size but not real inference time.
- Some methods may improve latency too much at the cost of unacceptable accuracy loss.
- Therefore, the final comparison will use a **trade-off perspective**, not a single metric winner.

Current status and discussion points

- Literature review and thesis framing are completed.
- Research questions and the comparative strategy are already defined.
- A prototype evaluation workflow and initial model assets are available in the repository.
- The next decision is to narrow the final experimental scope:
  - final dataset split,
  - baseline detector,
  - target hardware/runtime,
  - which combined pipeline is realistic within the thesis schedule.
- The plan is concrete enough to defend, but still open to improvement.

Summary

- The thesis objective is now defined precisely around the edge-deployment trade-off.
- The selected methodology is an experimental comparative study.
- The research design covers data, candidate models, parameters, and selection criteria.
- The planned architecture and implementation pipeline are clear.
- Validation will rely on practical deployment metrics, especially accuracy, latency, and size.

References

[1] Areeba Abid et al. **"Optimizing Medical Image Classification Models for Edge Devices".** In: *Distributed Computing and Artificial Intelligence, Volume 1: 18th International Conference*. Ed. by Kenji Matsui et al. Vol. 327. Lecture Notes in Networks and Systems. Cham: Springer, 2022, pp. 77–87. DOI: 10.1007/978-3-030-86261-9_8.

[2] Vivienne Sze et al. **"Efficient processing of deep neural networks: A tutorial and survey".** In: *arXiv:1703.09039* (2017).

[3] Tsung-Yi Lin et al. **"Microsoft COCO: Common Objects in Context".** In: *European Conference on Computer Vision (ECCV)*. 2014.

[4] Joseph Redmon et al. **"You Only Look Once: Unified, Real-Time Object Detection".** In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016.

[5] Mingxing Tan, Ruoming Pang, and Quoc V. Le. **"EfficientDet: Scalable and Efficient Object Detection".** In: *arXiv preprint arXiv:1911.09070* (2020).

[6] Andrew G. Howard et al. **"MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications".** In: *arXiv preprint arXiv:1704.04861* (2017).

[7] Wei Liu et al. **"SSD: Single Shot MultiBox Detector".** In: *European Conference on Computer Vision (ECCV)*. 2016.

References iii

[8] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. **"Distilling the knowledge in a neural network".** In: *arXiv:1503.02531* (2015).

[9] Nicolas Carion et al. **"End-to-End Object Detection with Transformers".** In: *European Conference on Computer Vision (ECCV)*. 2020.

[10] Amir Gholami, Sehoon Kim, Zhen Dong, et al. **"A survey of quantization methods for efficient neural network inference".** In: *arXiv:2103.13630* (2021).

[11] Song Han, Huizi Mao, and William J. Dally. **"Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding".** In: *ICLR*. 2016.

[12] Yu Cheng et al. **"A survey of model compression and acceleration for deep neural networks".** In: *arXiv:1710.09282* (2017).

[13] Han Cai, Ligeng Zhu, and Song Han. **"ProxylessNAS: Direct neural architecture search on target task and hardware".** In: *ICLR*. 2019.

Thank you for your attention!